

文件操作

字符串

```
name = "石磊"  
res = name.encode("utf-8")  
print(res) # b'\xe7\x9f\xb3\xe7\xa3\x8a'
```

字节

```
ress = res.decode("utf-8")  
print(ress) # 石磊
```

1.1 读文件

读文本文件

```
# 打开文件  
# -路径:  
#     相对路径: 'info.txt'  
#     绝对路径: 'User/24627/PythonData/info.txt'  
# -模式:  
#     rb: 表示读取文件原始的二进制 (r: 读read, b: 二进制)  
#     rt: 表示读取文件的text模式 (r: 读read, t: text)  
  
file=open('alice.txt',mode='rb')  
data=file.read()  
file.close()  
print(data)
```

判断文件是否存在

- 如果文件存在返回True, 否则返回False (相对路径绝对路径均可)

```
import os  
exists=os.path.exists('info.txt')  
print(exists)
```

- 代码实现
 - 使用exists判断文件是否存在, 如果存在则打开文件并读取文件, 不存在则提示文件不存在!!!

```

files = 'alice.txt'
import os

exists = os.path.exists(files)
if exists:
    file = open('alice.txt', mode='rt', encoding='utf-8')
    data = file.read()
    file.close()
    print(data)
else:
    print("文件不存在!!!")

```

1.2 写文件

- 写文件操作
 - 方法一:

```

# 打开文件
# -路径:
#     相对路径: 'info.txt'
#     绝对路径: 'User/24627/PythonData/info.txt'
# -模式:
#     wb: 表示读取文件原始的二进制 (w: 读 write, b: 二进制)
file = open('alice.txt', mode='wb')
file.write('石磊'.encode('utf-8'))
file.close()

```

- 方法二:

```

file = open('alice.txt', mode='wt', encoding='utf-8')
file.write('石磊')
file.close()

```

- 写文件之图片

```

file1 = open('a.jpg', mode='rb')
png = file1.read()
file1.close()

file2 = open('a2.png', mode='wb')
file2.write(png)
file2.close()

```

1.3 案例

先要安装第三方的模块

在Python终端中安装

pip install requests

一、在网上下载一些文本信息，将这些信息写入到文件中

```
import requests
# 将在网上下载的文本写入到文件中
res=requests.get(url='https://www.gov.cn/zhengce/zhengceku/2023-04/26/content_5753299.htm')
# get 为requests的使用方法
file1=open('案例1.txt',mode='ab')
file1.write(res.content)
# content为编码问题
file1.close()
```

二、在网上下载图片，将图片写入到文件中

如上，只是将URL的地址改为需要下载的图片地址，并且将文件名的后缀改为png
如要下载视频也是同样操作

三、使用文件操作中的a模式进行多用户的信息存储

```
file = open('user.txt', mode='wt')
while 1:
    user = input("user:")
    if user.upper() == 'Q':
        break
    pwd = input("pwd:")
    data = "{}-{}\n".format(user, pwd)
    file.write(data)
file.close()
```

1.4 常见用法

- read 用法
 - 读所有

```
file=open('txt.txt',mode='r')
file.read()
```

- 读字符

```
file=open('txt.txt',mode='r')
file.read(1)
```

- 读字节

```
file=open('user.txt',mode='rb')
res=file.read(1)
```

- readline 只读取第一行数据

```
file=open('txt.txt',mode='r')
file.readline()
```

- readlines 读取文件的所有行，并将数据作为一个列表

```
file=open('txt.txt',mode='r')
file.readlines()
```

- for循环读取 每一行输出

```
file=open('user.txt',mode='rt')
for i in file :
    print(i.strip())
```

- 查看值的内存地址

```
aa = 1
print(id(a))
# 使用id这个方法来看值的内存地址
```

ini 文件操作

```
import configparser # 导入的模块

# 1、获取文件中的节点内容
config = configparser.ConfigParser() # 必要的
config.read('my.ini', encoding='utf-8')
res = config.sections() # 各个节点的值
print(res)

# 2、获取mysqld节点下的值
ress=config.items('mysqld')
print(ress)
# # 直接输出
for k,v in ress:
    # 通过k, v的方式输出
    print(k+'==='+v)

# 3、获取某个节点下的键对应的值
result=config.get('mysqld','log-error')
# 上述方法的第一个值为节点名，第二个值为节点里面的键名
print(result)

# 4、其他功能
# 4、1 这个节点是否存在
v1 = config.has_section('client')
print(v1)
```

```
# 4、2 添加一个节点
config.add_section('group')
# 直接添加节点并在节点下添加键值
config.set('group', 'name', 'shilei')
config.write(open('my.ini', mode='w', encoding='utf-8'))

# 4、3删除一个节点
config.remove_section('client')
# 删除节点下面的键值,通过节点和对应的键来删除
config.remove_option('group', 'name')
config.write(open('my.ini', mode='w', encoding='utf-8'))
```

函数

1.1返回值

- 函数的返回值

- *args 任意参数

如果有多个参数传递, 返回的是元组类型

- **kwargs 任意的关键字参数

返回的是字典类型

- return

在函数内部使用return, 不管后面有没有其他代码, 只会执行到这里直接跳出函数

使用break不同, break只是会跳过当前的代码, 后面的代码还是会继续执行的

1.2 动态参数

- 形参固定, 实参用 *和**

```
def func(x,y):
    print(x,y)
func(1,3)
func(x=11,y=22)

# 下述的使用类型也是可以的
func(*[11,33])
func(**{'x':11, 'y':22})
```

- 形参用 *和**, 实参也用 *和**

```
def func(*args,**kwargs):
    print(args,kwargs)

func(1,3)
func(11,33,'name'='shilei','age'=23)

func([11,22,33],{'name'='shilei','age'=23})
func(*[11,22,33],**{'name'='shilei','age'=23})
```

1.3 函数做元素

```
data_list = ['shilei', 'func', func, func()]
print(data_list[0]) # shilei
print(data_list[1]) # func
print(data_list[2]) # <function func at 0x000001D047A949A0>
print(data_list[3]) # 123

res = data_list[2]() # 执行函数func，并获取函数值，print再输出返回值
print(res) # 123
```

1.4 作用域

1.4.1 在python中，是以函数来作为作用域的

```
def infi():
    for i in range(10):
        print(i)
    print(i)
# 在上述代码中，分别打印了两个i的值，但是程序是没有报错的
```

1.4.2 并且全局变量的参数名要大写

```
NAME = 'shilei'
AGE = '15'
def info():
    for i in range(len(AGE)):
        print(i)
info()
# 在上述带阿米中，在函数体外的代码依旧能被函数内使用，为全局变量
```

1.4.3 global全局变量

默认情况下，在局部作用域对全局变量只能进行：读取、修改内部元素的操作，无法对全局变量进行重新赋值

```
# 读取全局变量
COUNTRY='中国'
CITY_WALK=['北京','上海','武汉']
def info():
    print(COUNTRY,CITY_WALK)
info()
```

```

# 修改内部元素
COUNTRY='中国'
CITY_WALK=['北京','上海','武汉']
def info():
    CITY_WALK.append('杭州')
    CITY_WALK[0]='南京'
    return CITY_WALK
print(info())

# 无法对全局变量进行修改
COUNTRY='中国'
CITY_WALK=['北京','上海','武汉']
def info():
    COUNTRY='中华人民共和国'
    # 不是对全局变量进行赋值，只是在局部作用域中又创建了一个局部变量
    # 实际上的全局变量COUNTRY的值并没有改变
    print(COUNTRY)
print(COUNTRY) # 中国
info() # 中华人民共和国

# 强行修改全局变量
COUNTRY = '中国'
CITY_WALK = ['北京', '上海', '武汉']
def info():
    global COUNTRY #需要使用global关键字进行修改，使用时只能先重新定义全局变量，再进行赋值
    COUNTRY = '中华人民共和国'
    global CITY_WALK
    CITY_WALK = ['成都', '重庆', '长沙']
    print(COUNTRY)
    print(CITY_WALK)
info()
print(COUNTRY) # 中华人民共和国
print(CITY_WALK) # ['成都', '重庆', '长沙']

```

函数嵌套

1.1 函数的作用域

- 优先在自己的作用域找，没有就去上级作用域
- 在作用域中寻找值时，要确保此次此刻的值是什么
- 分析函数的执行，并确保函数的作用域（函数嵌套）

```

def fun(name=None):
    if not name:
        name = '石磊'
    def inner():
        print(name)
        return 'shieli'
    return inner
v1 = fun('aaa')()
# 由于fun函数返回的是inner，inner在fun函数里面是存在的，

```

```
# 故在fun('aaa')后面加上()就等于再次调用inner函数
# 所以就将shilei赋值给了v1
v2 = fun()()
# 这里是因为没有传入参数，所以默认赋值为 石磊
print(v1)
print(v2)
```

requests方法的请求头

```
headers = {
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36',
    'Accept':
    'text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8',
    'Accept-Language': 'en-US,en;q=0.9',
    'Authorization': 'Bearer your_access_token', # 替换为实际的访问令牌
    'Content-Type': 'application/json', }
```

```
headers = {
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36',
    'Accept':
    'text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8',
    'Accept-Language': 'en-US,en;q=0.9',
}
```

装饰器

- 视线原理：基于@语法和函数闭包，将函数封装在闭包中，然后将函数赋值为一个新的函数（内层函数），执行函数式再将内层函数中执行闭包中的原函数
- 实现效果：可以再不改变原有函数的内部代码和调用方法的前提下，实现在函数执行和执行扩展功能
- 使用场景：多个函数系统统一在执行前后自定义一些功能

代码实例：

```
def outer(orgain):
    def inner(*args,**kwargs):
        print('before')
        res = orgain(*args,**kwargs)
        print('after')
        return res
    return inner
@outer
# 上述代码的意思是： func=outer(func)
def func(*args,**kwargs):
    print('这是func函数',args,kwarg)
    for i in args:
        print(i)
    value = [11, 22, 33]
    return value
@outer
```



```
def func3(*args,**kwargs):  
    print(args,kwargs)  
func(1,2,a=5)  
func3(1,a2='shilei')
```

匿名函数 (lambda)

基于lambda定义函数格式为 `lambda 参数: 函数体`

- 参数，支持任意参数

```
lambda x : 函数体  
lambda x,y : 函数体  
lambda *args,**kwargs : 函数体
```

- 函数体，只能支持单行代码

```
lambda x:x+100  
# 相等于  
# def x():  
#     return x+100
```

- 返回值，默认将函数体单行代码的结果返回给函数的执行

```
func = lambda x:x+100  
res=func(101)  
print(res)
```

三元运算符

```
name = input('输入name: ')  
data = 'yes' if 'king' in name else 'no'  
print(data)  
  
# 相等于:  
# if 'king' in name:  
#     print('yes')  
# else:  
#     print('no')
```

格式为 `结果 = 条件成立时 if 条件 else 不成立`

使用三元运算符结合lambda

```
res = lambda x: '小了' if x < 66 else '大了'  
aa = res(100)  
print(aa)
```

生成器

生成器是由函数加上yield关键字创造出来的写法，在特定的情况下，用它可以帮助我们节省内存

- 生成器函数，当函数中有yield存在时，这个函数就是生成器函数
- 使用了生成器的函数，当这个函数被执行时，使用下述方法是不会执行函数内部的内容的，只有使用next()函数才会执行函数内部的内容，并且执行到yield这里就不会执行，如果有参数接收next的话，yield后面的内容就会赋值给那个参数，如下面的实例所示：

```
def func():  
    print(111)  
    yield 222  
aa = func()  
print(aa) # <generator object func at 0x000001D598624880>  
bb = next(aa) # 111  
print(bb) # 222
```

具体代码如下：

```
def func():  
    print(111)  
    print(222)  
    yield 123 # 有点像函数里的return，一次执行执行到这个位置就不会继续执行了  
    print(444)  
    yield 456 # 有点像函数里的return，一次执行执行到这个位置就不会继续执行了  
    print(666)  
    # 最后没有接yield，就相当于 return None  
    # 此时程序会报错，提示生成器中的代码执行完毕了 StopIteration  
aa = func()  
print(aa) # 执行生成器函数时，函数体默认不会被执行；返回的是一个生成器对象  
          # <generator object func at 0x000001E510724880>  
  
a = next(aa) # next里面放生成器对象，进入生成器函数并执行里面的代码  
print(a)  
  
b = next(aa) # 这次这个next只会从上次的yield返回位置继续向下执行，不会全部执行  
print(b)  
  
c = next(aa) # 如上，但是由于最后没有yield，所以执行带最后会报错 StopIteration  
print(c)  
  
# 如果使用for来遍历这个生成器函数，则最后不会报错  
data = func()  
for i in data:  
    print(i)
```

1.1 生成器应用场景

- 假设要生成300W个随机的四位数，并打印

```

import random
def get_random_num(max_num): # 形参为实际需要生成的数量
    count = 0 # 为计数用
    while count < max_num:
        print(count)
        yield random.randint(1000, 9999)
        count += 1 # 每次执行完后，计数参数+1
num = get_random_num(3000000)
n1 = next(num)
print(n1)
n2 = next(num)
print(n2)

```

1.2 带有参数的yield

```

def func():
    print(111)
    v1 = yield 1
    print(v1)

    print(222)
    v2 = yield 2
    print(v2)

    print(333)
    v3 = yield 3
    print(v3)
data = func()

n1 = data.send(None) # 111
print(n1) # 1

n2 = data.send(666) # 此时将携带的 666 重新赋值给yield，重新打印 v1，
                    # 往下执行打印 222，遇到了yield 将现在的 2 返回给 send 也就是n2，并
                    # 打印n2也就是 2，
print(n2)

n3 = data.send(888)
print(n3)

```

内置函数

- abs 绝对值

```
v = abs(-10)
```

- pow 指数

```
v1 = pow(2,5) # 2的5次方
```

- sum 求和

```
v1 = sum([1,2.3,4,5,6,-10]) # 可以被迭代 也就是for循环
```

- divmod 求商和余数

```
v1 , v2 = divmod(9,2) # v1指商 v2指余数
```

- round 小数点后n位 (四舍五入)

```
v1 = round(3.1415926,2) # round中的第一个代表执行的数，后面为需要保留的小数位
```

- min 最小值

```
v1 = min(-1,3,5,8,0)
```

```
v1 = min([11,22,33,44,55]) # 迭代的类型 (for循环)
```

```
v1 = min([-11, 22, 55, 44, 1], key=lambda x: abs(x))
```

- max 最大值

```
v1 = max(100,-23,4,78,99)
```

```
v1 = max([11,22,33,44,55]) # 迭代的类型 (for循环)
```

```
v1 = max([-11, 22, 55, 44, 1], key=lambda x: x * 10 )  
# 此时打印不会打印后面执行函数后的值，返回的是列表里面没有进行操作的值
```

- all 是否全部为True

```
v1 = all([11,22,3,0])
```

- any 是否存在True

```
v1 = any([-1,3,-4,0])
```

- bin 十进制转换为二进制
- oct 十进制转换为八进制
- hex 十进制转换为十六进制
- ord 获取字符对应的Unicode码点 (十进制)

```
v1 = ord('石')  
print(v1, hex(v1))
```

- chr 根据码点 (十进制) 获取对应的字符

```
v2 = chr(30707)  
print(v2)
```

- int 整型
- float 浮点型
- str unicode编码
- bytes utf-8, GBK编码

```
a = '石磊'
b = a.encode('utf-8')
print(b)
```

- bool 布尔型
- list 列表
- dict 字典
- tuple 元组
- set 集合
- len 获取长度
- print 输出
- input 输入
- open 打开文件
- type 获取类型
- range

```
for i in range(1,10):
    print(i)
```

- id 获取内存地址 (十进制)
- hash 哈希

```
name = hash('石磊')
print(name)
```

- help 获取帮助信息

```
import random
help(random)
```

- zip

```
a = [1, 2, 3, 4, 5, 6]
b = [11, 22, 33, 44, 55, 66]
c = [111, 222, 333]
res = zip(a, b, c)
for i in res:
    print(i)
```

- enumerate

```
name = ['shi', 'lei', 'king', 'bob']
for k, v in enumerate(name, 1):
    print(k, v)
```

- callable 是否可执行，后面是否可以加括号（返回的是false或者true)

```
a = 'aa'
b = lambda x: x
def c():
    pass
print(callable(a)) false
print(callable(b)) true
print(callable(c)) true
```

- sorted 排序

```
a = [11, 55, 99, 66, 12, 285, 46]
res = sorted(a, reverse=True) # 加了reverse为降序，不加则是升序
print(res)

name = {
    'shi': {
        'age': 23,
        'address': 'shanghai'
    },
    'lei': {
        'age': 25,
        'address': 'wuhan'
    },
    'admin': {
        'age': 1000,
        'address': 'china'
    }
}
res = sorted(name.items(), key=lambda x: x[1]['age'], reverse=True)
# 上述的key为固定用法，key=lambda x: x[1]['age'] 意思是 以字典的值（键的下标是0），值中的'age'进行排序
print(res)
```

推导式

通过一行代码实现创建list、dict、tuple、set并初始化一些值

1.1 列表

```
list = [i for i in range(10) if i > 5]
print(list)
```

1.2 字典

```
dict = {i: i for i in range(5)}
print(dict)
```

1.3 集合

```
set = {(i) for i in range(5)}  
print(set)
```

1.4 元组

```
tuple = ((i, i) for i in range(10) if i < 4)  
# 需要注意的是，元组不是直接返回结果，而是得到一个生成器  
for i in set:  
    print(i)
```

2.1 练习

2.1.1 去除列表中的后缀 (.txt)

```
data_list=[  
    'aljkghjgj alikjhg_edk.txt',  
    'asalkjhghjk_askgj.txt',  
    '123456782_edgdrg.txt'  
]  
res = [i.split('.',1)[0] for i in data_list]  
# 这里的 1 代表的是最大分割一次  
print(res)
```

2.1.2 将字典中的元素按照 键-值 格式化，并最终使用 ; 连接起来

```
data_dict = {  
    'name': 'shilei',  
    'age': 23,  
    'address': 'China'  
}  
res = ';'.join(['{}-{}'.format(k, v) for k, v in data_dict.items()])  
print(res)
```

模块

1 包和模块

包是有多个py文件组成的文件夹，并且需要有 `__init__.py` 这个文件（哪怕里面啥都没有，如果里面有代码的话，导入包下面的模块时也会一起执行这个 `__init__.py` 里面的内容）

对于python2是这样的

但是python3的话可有可无，若有就运行，没有就不运行

导入模块只会在当前路径寻找

如果需要导入的模块不在当前路径（如果不在当前寻找的路径列表里话，是不会导入成功的）

则需要：

```
import sys
# 先导入sys
sys.path.append(r'D:\xxx\xxx')
# 将需要添加的模块的文件路径添加到python内部寻找的路径的列表里
from xxx import xxx
# 再来导入需要的包

# 例如：
import sys
sys.path.append(r'C:\Users\24627\Desktop\Python_Study\StudyPython\test_bao')
from test_bao.MD5 import md5sum
```

2 内置模块

2.1 os模块

2.1.1 listdir 查看文件夹下面的文件（不会显示详细文件）

```
import os
res = os.listdir(r'C:\Users\24627\Desktop\Python_Study\StudyPython')
print(res)
# ['.idea', 'fromemail.py', 'Python.md', 'test_bao', '__init__.py',
#  '__pycache__', '函数', '文件操作', '烟花.py']
# 只会显示里面的内容，不会递归显示
```

2.1.2 walk 查看文件夹下面的所有文件（递归显示）

```
# 显示Desktop\Python_Study\StudyPython下的所有带py的文件
import os
res = os.walk(r'C:\Users\24627\Desktop\Python_Study\StudyPython')
print(res)
# 不会打印内容，只是一个生成器
for path, file, data in res:
# path代表的是路径，file是文件夹，data是文件
    for i in data:
# 将所有文件打印
        result = os.path.join(path, i)
# 将路径与文件连接起来
        aa = result.split('.', 1)[-1]
# 取文件的后缀
        if aa == 'py':
            print(result)
```


2.2 random模块

```
import random

a = random.randint(1, 10)
print(a)
# 获取随机数
b = random.uniform(1, 10)
print('%.4f' % b)
# 获取小数, 并保留4位
c = random.choice([11, 22, 33, 44, 55, 66, 77, 88, 99])
print(c)
# 在列表里随机一个数
d = random.sample([11, 22, 33, 44, 55, 66, 77, 88, 99], 3)
print(d)
# 在列表里随机多个数
data = [11, 22, 33, 44, 55, 66, 77, 88, 99]
random.shuffle(data)
print(data)
# 随机打乱顺序
```

2.3 json模块

json模块, 可以将Python的数据格式转换为json格式的数据, 也可以将json格式转换成Python格式

`json.dumps` 序列化生成一个字符串 (Python-->json)

`json.loads` 序列化生成Python数据类型 (json-->python)

```
import json

data = [{ 'name': '石磊', 'age': 23, 'address': 'hubei' }, { 'id': 3 }]

res = json.dumps(data)
res = json.dumps(data, ensure_ascii=False)
# ensure_ascii=False 将unicode编码变成utf-8编码
print(res)

result = json.loads(res)
print(result)
```

`json.dump` 将数据序列化并写入文件

```
import json

data = [{
    'id': 2,
    'name': '石磊',
    'age': 34
},
{
    'id': '001',
    'name': 'shilei',
```

```

        'age': 18
    }
]
file = open('json_data.json', mode='w', encoding='utf-8')
json.dump(data, file)
file.close()

```

`json.load` 读取文件中的数据并反序列化为Python数据类型

```

import json

file = open('json_data.json', mode='r', encoding='utf-8')
data = json.load(file)
print(data)
file.close()

```

2.4 时间模块

- time: 时间戳 (自1970-1-1 00:00 开始)
- datetime: 获取当前本地时间
- sleep: 暂停几秒
- timezone: 以时区来显示时间
- strftime: 将当前时间的类型转换成字符串 ('%Y--%m--%d %H:%M:%S')为固定搭配
- datetime.fromtimestamp: 将时间戳时间转换成datetime格式
- timestamp: 将datetime转换成时间戳格式

```

import time
bb = time.time()
print(bb)

from datetime import *
aa = datetime.now()
print(aa)

print('开始')
time.sleep(3)
print('结束')

cc = timezone(timedelta(hours=8))
dd = datetime.now(cc)
print(dd)

aaa = aa.strftime('%Y--%m--%d %H:%M:%S')

bbb = datetime.fromtimestamp(bb)
print(bbb)

aaaa = aa.timestamp()
print(aaaa)

```

2.5 模块练习

- 在日志中将时间和自定义输入的内容作为标题

```
from datetime import *
def log():
    log_name = input('请输入log标题: ')
    if log_name.upper() == 'Q':
        print('error')
    name = input('请输入日志内容:')

    tiems = datetime.now().strftime('%Y-%m-%d--%H: %M: %S')
    file = '{}{}.txt'.format(log_name, tiems)
    with open(file, mode='a', encoding='utf-8') as file_name:
        file_name.write(name)
        file_name.flush()
if __name__ == '__main__':
    log()
```

- 用户注册，将用户信息写入Excel，其中包括：用户名、密码、注册时间。

3 正则表达式

3.1 re模块

3.1.1 字符相关

- `shilei` 匹配文本中的shilei

```
import re

text = '.skjkjsjknfjshileikdjnfjshileidknjshieileishi'
data = re.findall('shilei', text)
print(data)
```

- `[abc]` `q[a-t]` 匹配a或者b或者c，匹配第一个为q第二个是a-t之间的。

```
import re

text = '.skjkjsjkaabbttndqfjqshqileikdjqnfsqhilqeidknjshieileishi'
data = re.findall('[abc]', text)
data1 = re.findall('q[a-t]', text)
print(data)
print(data1)
```

- `[^abc]` 匹配除了abc以外的字符

```
import re

text = '.skjkjsjkaabbttndqfjqshqileikdjqnfsqhilqeidknjshieileishi'
data2=re.findall('[^abc]',text)
print(data2)
```

- `[a-z]` 匹配a-z的任意字符
- 除换行符以外的所有字符

1、`s.j` 匹配s j的字符

```
import re

text = '.skjkjseeejkaabbttndqfjqshqilseqikdjsqnfjsqhilqesidknjshieileisshi'
data = re.findall('s.j', text)
print(data)
```

2、`s.+j` 贪婪匹配

直接匹配开头是s结尾是j的字符

```
import re

text = '.skjkjseeejkaabbttndqfjqshqilseqikdjsqnfjsqhilqesidknjshieileisshi'
data = re.findall('s.+j', text)
print(data)
# ['skjkjseeejkaabbttndqfjqshqilseqikdjsqnfjsqhilqesidknj']
```

3、`s.+?j` 非贪婪匹配

将中间有其他符合条件的打印

```
import re

text = '.skjkjseeejkaabbttndqfjqshqilseqikdjsqnfjsqhilqesidknjshieileisshi'
data = re.findall('s.+?j', text)
print(data)
# ['skj', 'seeej', 'shqilseqikdj', 'sqnfj', 'sqhilqesidknj']
```

- `r'\w'` 代指字母或数字或下划线（汉字）

```
import re
text = '.s11kjkjs99eeejkaab石数据bt石开始的tndqfjqs7hqilseq石都费劲
ikdttjs3qnfjsqhilqesidknjshieileis0s0hi'
data1 = re.findall(r"石\w+b", text)
print(data1)
```

- `r'\d'` 代指数字

```
import re
text = '.s11kjkjs99eeejkaab石数据bt石开始的tndqfjqs7hqilseq石都费劲
ikdttjs3qnfjsqhilqesidknjshieileis0s0hi'
data2=re.findall(r's\d+',text)
print(data2)
```

- `r'\s'` 代指任意的空白符，包括空格，制表符等

```
text = '.s11 kjkjs99eeejkaab石数据bt石开始的tndqfjqs7hqilseq石都费劲
ikdttjs3qnfjsqhilqesidknjshieileis0s0hi '
data3=re.findall(r's\d+\s\w',text)
print(data3)
```

3.1.2 数量相关

- * 重复0次或多次

```
test = '98他765他45678他90987他65678他909876他54678他98765'
data11 = re.findall(r'他\d*', test)
print(data11)
```

- + 重复1次或多次

```
test = '98他765他45678他90987他65678他909876他54678他98765'
data11 = re.findall(r'他\d+', test)
print(data11)
```

- ? 重复0次或1次

```
test = '98他765他45678他90987他65678他909876他54678他98765'
data11 = re.findall(r'他\d?', test)
print(data11)
```

- {n} 重复n次

```
test = '98他765他45678他90987他65678他909876他54678他98765'
data11 = re.findall(r'他\d{2}', test)
print(data11)
```

- {2,} 重复n次或多次

```
test = '98他765他45678他90987他65678他909876他54678他98765'
data11 = re.findall(r'他\d{2,}', test)
print(data11)
```

- {n,m} 重复n到m次

```
test = '98他765他45678他90987他65678他909876他54678他98765'
data11 = re.findall(r'他\d{2,4}', test)
print(data11)
```

3.1.3 括号 (分组)

- 提取数据区域

```
test = '1234567890987654321234567890'
res = re.findall(r'123(4\d{3})', test)
print(res)
```

```
test = '1234567890987654321234567890'
res = re.findall(r'1(23)(4\d{3})', test)
print(res)
# [('1234567', '23', '4567'), ('1234567', '23', '4567')]
```

- 获取指定区域+或条件

```
test = '12345678123467899098765432'
res1 = re.findall(r'(1234(5\d+?|6\d{3}))', test)
print(res1)
# [('123456', '56'), ('12346789', '6789')]
```

3.1.4 起始和结束

进行编写正则表达式时，必须严格遵守起始和结束的字符

- `^` 起始
- `$` 结束

```
mail = '12344546@qq.com和XXXXXX@live.com'
mail1 = '123456@qq.com'
res = re.findall(r'^\w+@\w+\.\w+$', mail, re.ASCII)
res1 = re.findall(r'^\w+@\w+\.\w+$', mail1, re.ASCII)
print(res)
# []
print(res1)
# ['123456@qq.com']
# 上述中，res进行匹配时由于结束不是由设定的字符结束，故没有匹配成功
```

3.1.5 特殊字符

由于正则表达式中的 `*` `.` `\` `{` `}` `(` `)` 都具有

特殊的意义，在使用的过程中需要进行转义。

使用 `\\` 进行

3.2 re模块拓展

- `match` 从其实位置进行匹配。匹配成功返回一个对象，未成功返回None

```
test = '111222333999888777'
data = re.match('222', test)
print(data)
# None
```

- `search` 浏览整个字符串进行匹配，未匹配成功返回None
 - 需要用`group`函数进行输出，否则返回的是一个生成器

```
test = '111222333999888777'
data1 = re.search('222', test)
print(data1.group())
```

- sub 替换匹配成功的字符，后面再加一个参数代表只匹配几次

```
test = 'aaaAAABBBBBB'
data2 = re.sub('B', 'b', test)
print(data2)

test = 'aaaAAABBBBBB'
data2 = re.sub('B', 'b', test, 1)
print(data2)
```

- split 根据匹配成功的字符进行分割

```
test = 'aaa1bbb1ccc1ddd'
data3 = re.split('1', test)
print(data3)
```

- finditer

```
test1 = '123456200005021010db134565419991210101x'
data4 = re.finditer(r'\d{6}(?P<year>\d{4})(?P<month>\d{2})(?P<day>\d{2})\d{3}[0-9|x]', test1)
for res in data4:
    result = res.groupdict()
    print(result)
```

面向对象

1、面向对象

- 定义类：在类中定义方法，在方法中去实现具体的功能
- 实例化类的一个对象，通过对象去调用并执行方法

```
class FromTo():
    def __init__(self, file):
        self.file = file

    def prints(self, email):
        data = '发送的邮箱是{}, 内容是{}'.format(email, self.file)
        print(data)

name1 = FromTo('Hello, world! ')
name1.prints('123456@qq.com')
```

需要注意的是：

- 1、类的首字母必须大写或者遵循驼峰命名
- 2、python3之后默认类都继承object
- 3、在类中编写的函数称之为方法
- 4、每个方法的第一个参数是self

1.1 对象和self

- 在每个类中都可以定义一个特殊的 `__init__` 初始化方法，在实例化类创建对象时自动执行，即 `对象=类()`

```
class Login():
    global name,password
    def __init__(self, name, password):
        self.name = name
        self.pwd = password

    def from_login(self):
        name_list = []
        while True:
            name = input('用户名: ')
            if name.upper() == 'Q':
                break
            password = input("密码: ")
            user_login = Login(name, password)
            name_list.append(user_login)
        for i in name_list:
            print(i.name, i.pwd)

user=Login(1,1)
# 对象=类名() 会自动执行类中的__init__方法

# 根据类型创建一个对象，是内存的一块区域
# 执行__init__方法，模块会将创建的那块区域的内存地址当self参数传递进去

user.from_login()
```

1.2 应用实例

```
class Police():
    def __init__(self, name, role):
        self.name = name
        self.role = role
        if role == '队长':
            self.heart = 500
        else:
            self.heart = 200
    def show_status(self):
        if self.heart == 0:
            print('{}你死了'.format(self.name))
        mess = '{}你的血量还剩{}'.format(self.name, self.heart)
        print(mess)
    def bobm(self, tree_list):
        for i in tree_list:
            i.blood -= 200
            i.show_status()

class Tree():
    def __init__(self, name, blood=300):
```



```

        self.name = name
        self.blood = blood
    def shoot(self, police_name):
        police_name.heart -= 20
        self.blood -= 5
        police_name.show_status()
    def shoot_status(self, police_list):
        for i in police_list:
            i.heart -= 15
            i.show_status()
    def show_status(self):
        ress = '{}你的血量还剩{}'.format(self.name, self.blood)
        print(ress)
def main():
    p1 = Police('shilei', '队长')
    p2 = Police('king', '队员')
    p3 = Police('aaa', '队员')
    t1 = Tree('1234')
    t2 = Tree('5678')
    t1.shoot(p1)
    t1.shoot_status([p2, p1, p3])
    p1.bobm([t1, t2])
if __name__ == '__main__':
    main()

```

2、三大特性

2.1 封装

- 将同一类方法封装到一个类中，如上述代码中的Police类和Tree类
- 将数据封装到对象中，再实例化一个对象，可以通过 `__init__` 初始化方法在对象中封装一些数据，便于以后使用

2.2 继承

- 需要两个类，分为父类和子类，也可以叫基类和派生类
- 子类可以继承父类中的方法和类变量（不是拷贝，父类的还是属于父类，子类可以继承（使用）而已）

```

class Base():
    def f1(self):
        print('before')
        self.f2()
        # self是obj对象（Foo类创建的对象）  obj.f2
        print('base.f1')
    def f2(self):
        print('base.f2')
class Foo(Base):
    def f2(self):
        print('foo.f2')
obj = Foo()
obj.f1()
# 优先去Foo类中找f1，因为调用f1的那个对象是Foo类创建的
# before

```

```
# foo.f2
# base.f1

b1=Base()
b1.f1()
# before
# base.f2
# base.f1
```

- 多继承的情况:

```
class Class1():
    def aa(self):
        print('Class1')
class Class2():
    def aa(self):
        print('Class2')
class User(Class1, Class2):
    def run(self):
        print('before')
        self.aa()
        print('after')
f1 = User()
f1.run()
# before
# Class1
# after
```

在上述的User类中，该类继承了两个父类，两个父类中存在相同的方法aa，在User类中，使用了该方法，多继承的顺序是先找第一个父类，如果第一个没有，就往后找，依次执行

```
class A():
    def aa_def(self, poll_interval=0.5):
        print('aa')
        self.bb_def()
    def bb_def(self):
        print('bb')
        self.cc_def(request=0, client_address=0)
    def cc_def(self, request, client_address):
        print('cc')
        pass
class TCPserver(A):
    pass
class Thread():
    def process_request(self, request, client_address):
        pass
class ThreadTcpserver(Thread, TCPserver):
    pass
obj=ThreadTcpserver()
obj.aa_def()
# aa
# bb
# cc
```

上述代码中，obj是ThreadTcpserver的对象，调用aa_def方法是，自己的类中是没有的，向第一优先级的类Thread中寻找，发现没有，且没有父类；第二优先级TCPserver总寻找，也没有，但是有父类A，在A类中找到aa_def方法，发现当中又调用了bb_def方法，又从ThreadTcpserver类开始寻找，又找到了A类，执行bb_def方法，当中又调用了cc_def方法，从头开始找，有时在A类中找到该方法，随即执行。

原则就是自己类中没有的就向上一级类找，但是遇到了self又要从调用的那个类重新开始调用这个方法

```
class A():
    def aa_def(self, poll_interval=0.5):
        print('aa')
        self.bb_def()
    def bb_def(self):
        print('bb')
        self.cc_def(request=0, client_address=0)
    def cc_def(self, request, client_address):
        print('cc')
        pass
class TCPserver(A):
    pass
class Thread():
    def process_request(self, request, client_address):
        pass
    def cc_def(self, request, client_address):
        print('this is Thread cc_def')
        pass
class ThreadTcpserver(Thread, TCPserver):
    pass
obj=ThreadTcpserver()
obj.aa_def()
# aa
# bb
# this is Thread cc_def
```

假如将上述代码改成Thread类中也有一个相同的类cc_def，则执行到bb_def方法后，向下寻找cc_def方法，在找到Thread类中有cc_def方法则执行Thread类中的cc_def方法。

2.3 多态

多态，其实就是多种形态

```
class A(object):
    def send(self):
        print('A_send')
class B(object):
    def send(self):
        print('B_send')
def user(arg):
    arg.send()
v1 = A()
user(v1)
```

```
v2 = B()
user(v2)

# A_send
# B_send
```

在程序设计中，鸭子类型（duck typing）是动态类型的一种风格。在鸭子类型中，关注点在于对象的行为，能做什么；而不是管住对象所属的类型

2.4 小结

- 封装：将方法封装到类中或将数据封装到对象中，便于以后使用
- 继承：将类中的公共方法提取到基类中去实现
- 多态：python默认支持多态（鸭子类型），最简单的基础下面代码

```
def func(aa):
    res = copy.copy(aa)
    print(res)

func('shilei')
func([1,2,3,4,5])
```